LAPORAN PRAKTIKUM

BASIS DATA

UNIT 11

Agregation



Disusun oleh:

Nama	: Azura Rizky Annisa
NIM	: 24/535715/SV/24290
Kelas	: R12B1
Dosen Pengampu	: Anni Karimatul Fauziyyah, S.Kom.,M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA INTERNET
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2025

A. Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut :

1. Mahasiswa mampu memahami konsep agregasi
2. Mahasiswa mampu memahami dan menggunakan jenis-jenis agregasi serta fungsi-fungsinya

B. Dasar Teori

Agregasi merupakan proses penting dalam pengelolaan basis data yang memungkinkan pengolahan data secara kompleks dan mendalam. Dalam konteks SQL dan MongoDB, agregasi digunakan untuk melakukan perhitungan, pengelompokan, dan transformasi data dari kumpulan dokumen atau tabel untuk mendapatkan informasi yang lebih berarti dan terperinci. Pada basis data SQL, fungsi agregasi seperti SUM(), AVG(), MAX(), MIN(), dan COUNT() sering digunakan untuk melakukan perhitungan berdasarkan satu atau lebih kolom dari sebuah tabel, sehingga memudahkan analisis dan pengambilan keputusan.

Sementara itu, dalam MongoDB, konsep agregasi diimplementasikan melalui metode khusus yang disebut Aggregation Framework. Framework ini memungkinkan pengguna melakukan berbagai tahap operasi pada dokumen, seperti penyaringan, pengelompokan, pengurutan, dan modifikasi data secara berulang-ulang dalam satu proses. Pada dasarnya, operasi agregasi di MongoDB berbasis pipeline yang terdiri dari beberapa tahap yang saling berhubungan, seperti $match, $group, $sort, dan $project. Pipeline ini memungkinkan analisis data yang lebih kompleks dan fleksibel dibandingkan dengan kueri biasa, sehingga cocok untuk kebutuhan analitik dan pengolahan data besar.

Secara umum, baik dalam SQL maupun MongoDB, agregasi sangat penting untuk membangun laporan, analisis data, dan pengambilan keputusan strategis. Kedua sistem ini menyediakan fungsi dan mekanisme yang kuat untuk mengelompokkan, merangkum, dan memanipulasi data secara efisien sesuai dengan kebutuhan pengguna. Dengan demikian, proses agregasi menjadi fondasi utama dalam melakukan pengolahan data yang lebih canggih dan informatif.

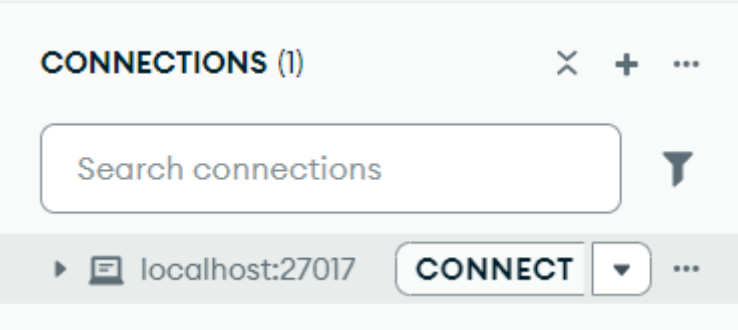
C. Alat dan Bahan

Alat dan bahan yang digunakan pada praktikum ini adalah sebagai berikut:

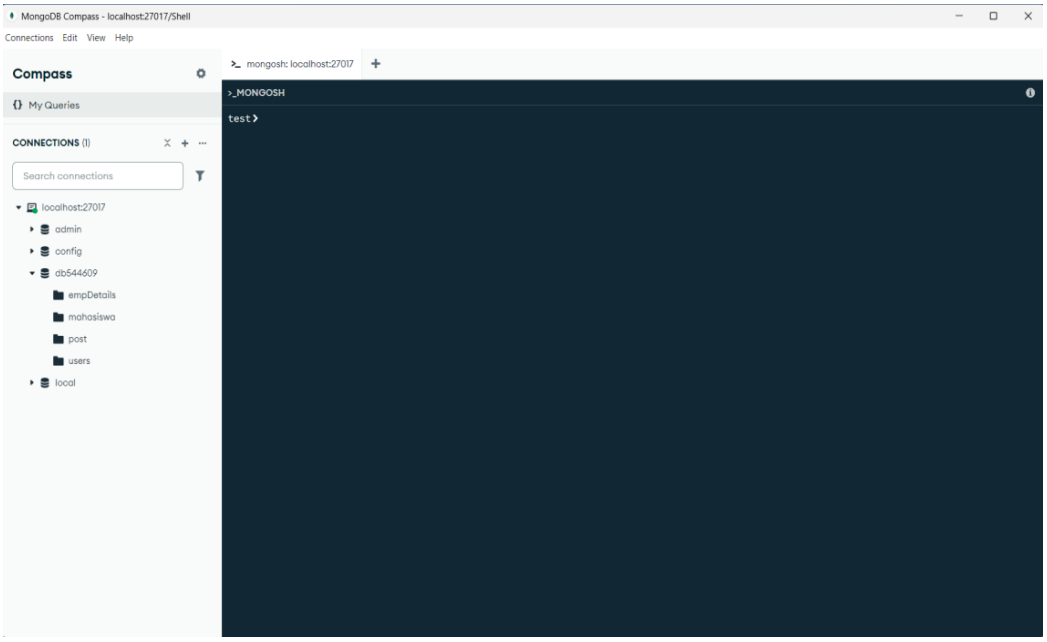
1. Laptop/Desktop/Komputer
2. MongoDBCompass

D. Prosedur Percobaan

1. Buka aplikasi MongoDBCompass (install terlebih dahulu aplikasi mongodb compass)
2. Klik tombol CONNECT di sebelah koneksi localhost:27017



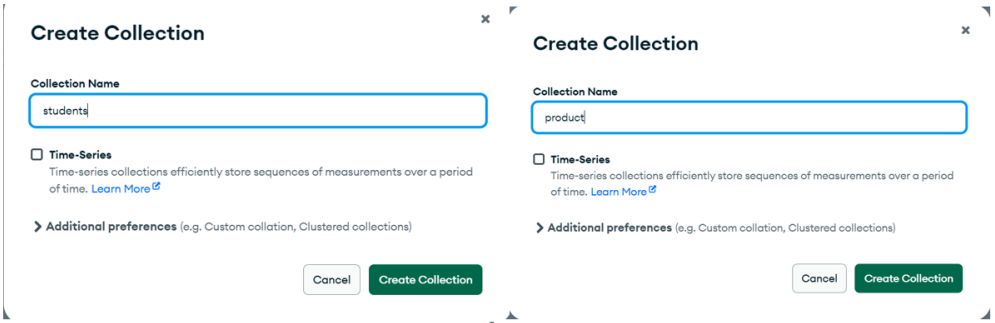
3. Masuk ke database yang sudah dibuat, atau membuat databaase baru jika belum membuatnya
4. Pilih fitur Open MongoDB Shell dan ketik perintahnya

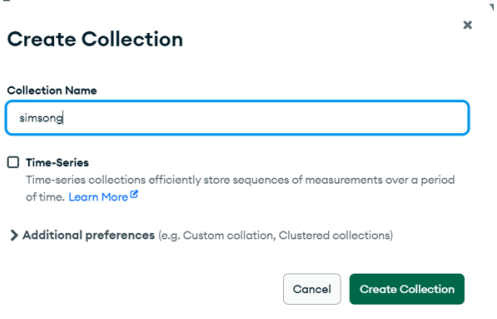


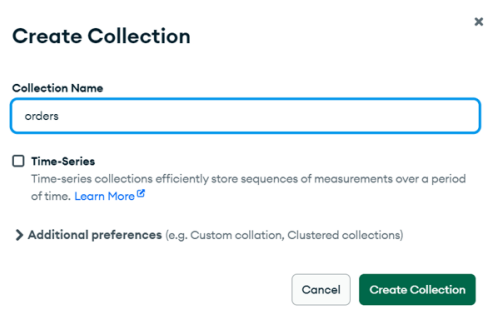
E. Hasil dan Pembahasan

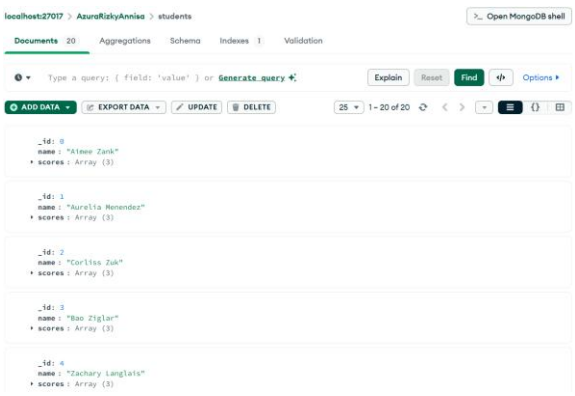
1. Import database

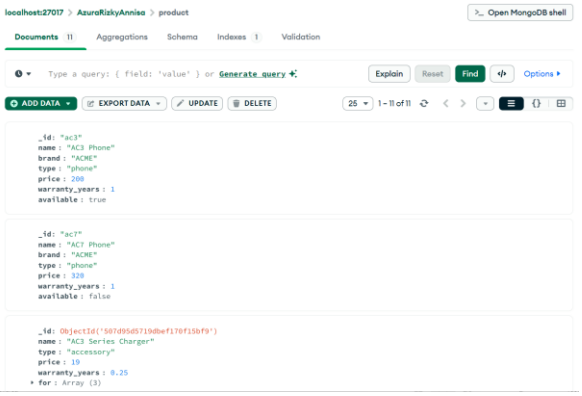
   Import database students.json dan order.json

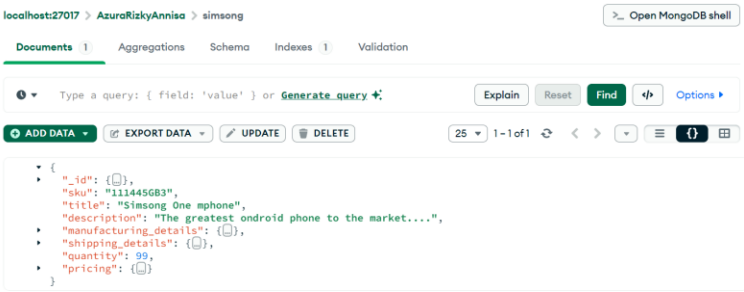












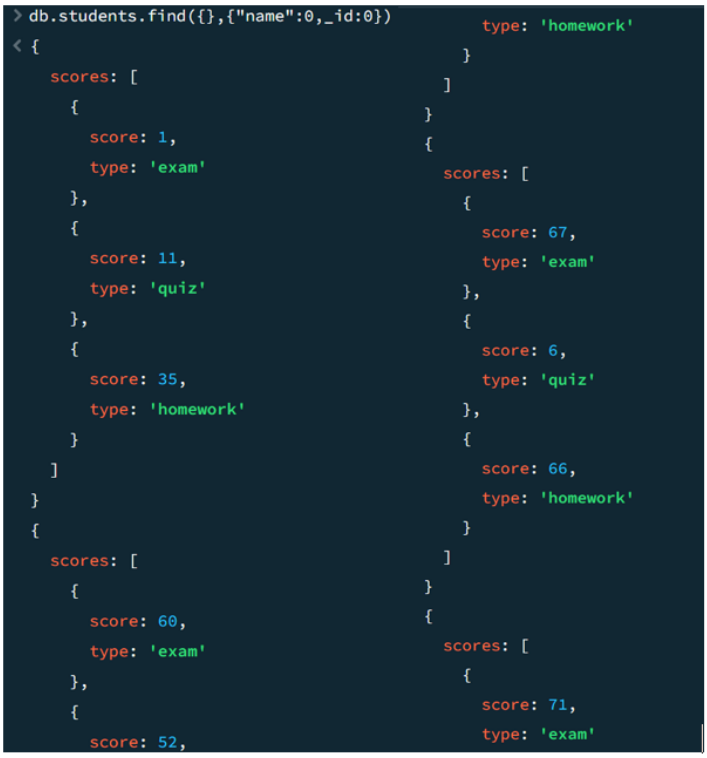
Pembahasan:

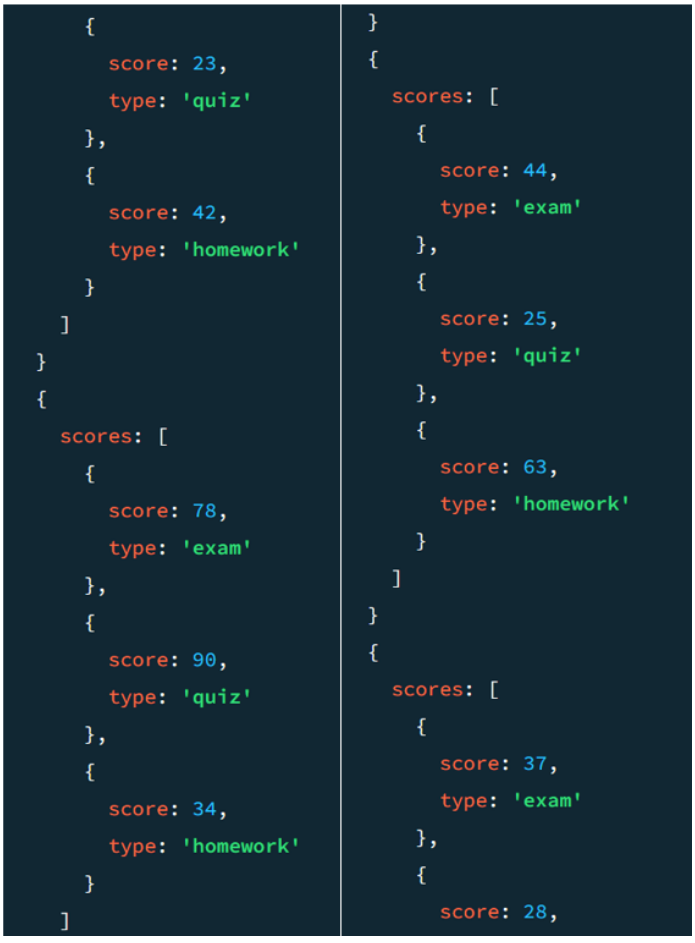
Import database students, product, dan order yang sudah di download terlebih dahulu. Pertama-tama buat database baru untuk setiap file di MongoDB, lalu import file JSON di masing-masing database. Pengimportan database ini bertujuan untuk menyediakan data yang akan digunakan praktik agregasi.

2. Jelaskan perbedaan key 1 atau 0 pada fungsi find()
   - Key 1

```
> db.students.find({}, { "name": 1, _id: 0 })
< {
    name: 'Aimee Zank'
  }
  {
    name: 'Aurelia Menendez'
  }
  {
    name: 'Corliss Zuk'
  }
  {
    name: 'Bao Ziglar'
  }
  {
    name: 'Zachary Langlais'
  }
  {
    name: 'Wilburn Spiess'
  }
  {
    name: 'Jenette Flanders'
  }
  {
    name: 'Salena Olmos'
  }
  {
    name: 'Daphne Zheng'
  }
  {
    name: 'Sanda Ryba'
  }
  {
    name: 'Denisha Cast'
  }
  {
    name: 'Marcus Blohm'
  }
  {
    name: 'Quincy Danaher'
  }
  {
    name: 'Jessika Dagenais'
  }
  {
    name: 'Alix Sherrill'
  }
  {
    name: 'Tambra Mercure'
  }
  {
    name: 'Dodie Staller'
  }
  {
    name: 'Fletcher Mcconnell'
  }
  {
    name: 'Verdell Sowinski'
  }
  {
    name: 'Gisela Levin'
  }
```

- Key 0

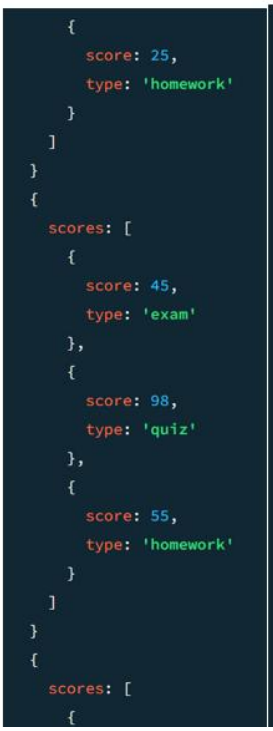

db.students.find({},{"name":0,_id:0})
{
scores: [
{
score: 1,
type: 'exam'
},
{
score: 11,
type: 'quiz'
},
{
score: 35,
type: 'homework'
}
]
}
{
scores: [
{
score: 60,
type: 'exam'
},
{
score: 52,
type: 'homework'
}
]
}
{
scores: [
{
score: 67,
type: 'exam'
},
{
score: 6,
type: 'quiz'
},
{
score: 66,
type: 'homework'
}
]
}
{
scores: [
{
score: 71,
type: 'exam'


{
score: 23,
type: 'quiz'
},
{
score: 42,
type: 'homework'
}
]
}
{
scores: [
{
score: 78,
type: 'exam'
},
{
score: 90,
type: 'quiz'
},
{
score: 34,
type: 'homework'
}
]
}
{
scores: [
{
score: 44,
type: 'exam'
},
{
score: 25,
type: 'quiz'
},
{
score: 63,
type: 'homework'
}
]
}
{
scores: [
{
score: 37,
type: 'exam'
},
{
score: 28,



```
      type: 'quiz'
    },
    {
      score: 81,
      type: 'homework'
    }
  ]
}
{
  scores: [
    {
      score: 90,
      type: 'exam'
    },
    {
      score: 42,
      type: 'quiz'
    },
    {
      score: 96,
      type: 'homework'
    }
  ]
}
{
```

```
  scores: [
    {
      score: 22,
      type: 'exam'
    },
    {
      score: 14,
      type: 'quiz'
    },
    {
      score: 75,
      type: 'homework'
    }
  ]
}
{
  scores: [
    {
      score: 97,
      type: 'exam'
    },
    {
      score: 97,
      type: 'quiz'
    },
```



```
    {
      score: 25,
      type: 'homework'
    }
  ]
}
{
  scores: [
    {
      score: 45,
      type: 'exam'
    },
    {
      score: 98,
      type: 'quiz'
    },
    {
      score: 55,
      type: 'homework'
    }
  ]
}
{
  scores: [
    {
```



```
      score: 78,
      type: 'exam'
    },
    {
      score: 82,
      type: 'quiz'
    },
    {
      score: 87,
      type: 'homework'
    }
  ]
}
{
  scores: [
    {
      score: 54,
      type: 'exam'
    },
    {
      score: 85,
      type: 'quiz'
    },
    {
      score: 80,
```



Pembahasan:

Fungsi find() digunakan untuk mengambil data dari koleksi di MongoDB. Parameter kedua dari fungsi ini adalah fitur projection, yaitu sebuah objek yang menentukan bidang-bidang mana saja yang akan ditampilkan atau disembunyikan dalam hasil pencarian. Nilai 1 menunjukkan bahwa bidang tersebut akan ditampilkan, sementara nilai 0 berarti bidang tersebut disembunyikan. Sebagai contoh, perintah db.student.find({}, {"name": 1, "_id": 0}) akan menampilkan hanya field name dan menyembunyikan field _id. Fitur ini sangat berguna saat kita hanya ingin melihat informasi tertentu dari dokumen tanpa harus menampilkan seluruh data yang ada.

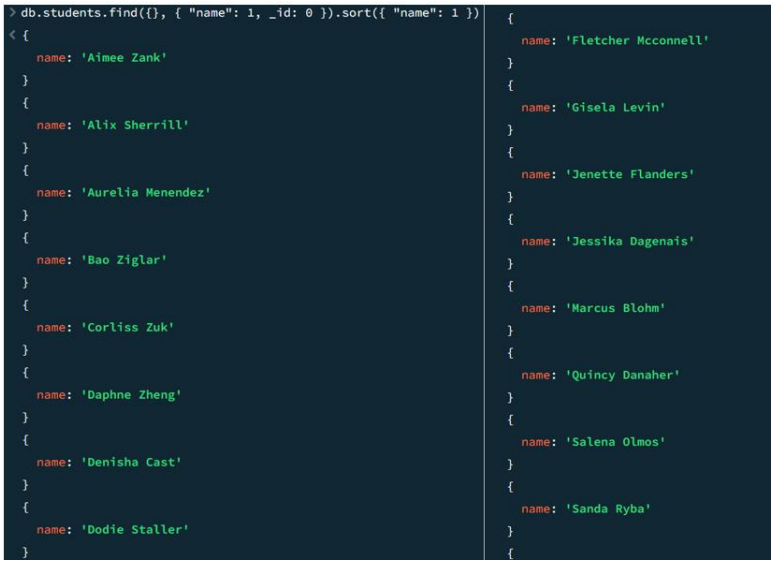
3. Jelaskan penggunaan fungsi skip

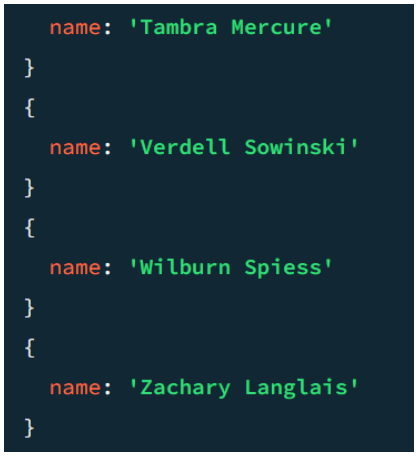


Pembahasan:

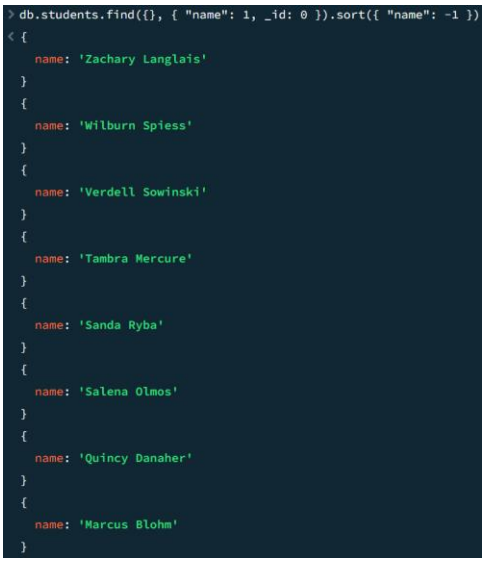
Pada hasil query di atas melakukan percobaan penggunaan fungsi skip(). Fungsi skip() digunakan untuk melewati sejumlah n dokumen pertama dari hasil pencarian dalam koleksi database. Pada query pertama dengan perintah db.student.find({},{"name":1, _id:0}).limit(5).skip(1) berarti sistem akan mengambil field name tanpa menampilkan field _id, lalu melewati 1 dokumen pertama dan hanya mengambil 5 dokumen berikutnya. Hasilnya adalah data dimulai dari nama “Aurellia Menendez” sampai “Wilburn Spiess”. Sedangkan pada query kedua db.student.find({},{"name":1, _id:0}).limit(5).skip(2) melewati 2 dokumen pertama dan mengambil 5 dokumen setelahnya, yaitu dari “Corliss Zuk” sampai “Jenette Flanders”.

4. Jelaskan penggunaan fungsi sort() 1 atau -1
   - Sort 1





- Sort -1





Pembahasan:

Pada gambar tersebut, terlihat contoh penggunaan fungsi sort(). Fungsi ini digunakan untuk mengurutkan dokumen hasil pencarian berdasarkan nilai dari sebuah field tertentu. Jika parameter diberikan dengan nilai 1, maka pengurutan dilakukan secara naik (ascending), dan jika diberi nilai -1, pengurutan dilakukan secara menurun (descending). Sebagai contoh, query pertama db.student.find({},{"name":1, _id:0}).sort({"name":1}) akan mengurutkan data berdasarkan field name secara alfabet dari A ke Z, mulai dari “Aimee Zank” hingga “Zachary Langlais”. Sebaliknya, query kedua db.student.find({},{"name":1, _id:0}).sort({"name":-1}) akan mengurutkan data secara terbalik dari Z ke A, mulai dari “Zachary Langlais” sampai “Aimee Zank”.

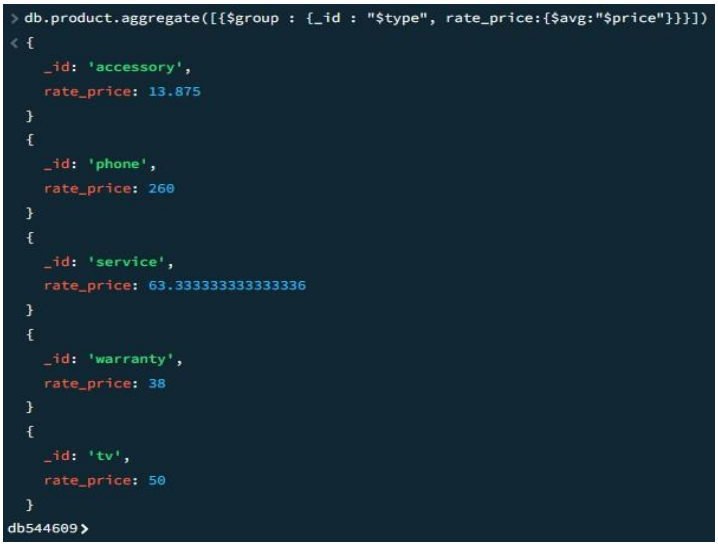
Untuk query berikut, gunakan collection product json

5. Jelaskan fungsi aggregasi berikut berdasarkan hasil query

```
> db.product.aggregate([{$group : {_id : "$type", num_product:{$sum:1}}}])
< {
    _id: 'phone',
    num_product: 2
  }
  {
    _id: 'accessory',
    num_product: 4
  }
  {
    _id: 'service',
    num_product: 3
  }
  {
    _id: 'tv',
    num_product: 1
  }
  {
    _id: 'warranty',
    num_product: 1
  }
db544609>
```

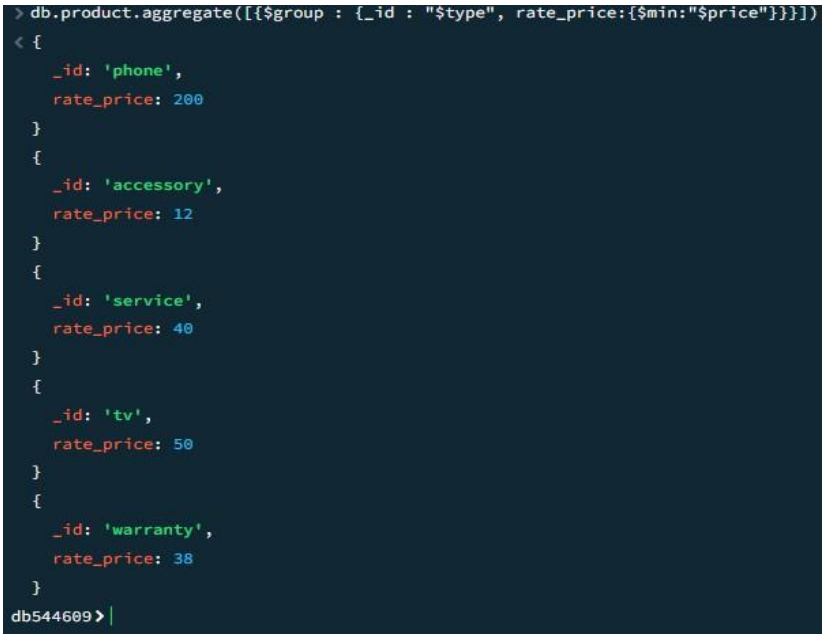
Pembahasan:

Dalam hasil query tersebut, dilakukan percobaan menggunakan fungsi aggregate() dengan tahap $group. Tahap $group berfungsi untuk mengelompokkan dokumen berdasarkan field tertentu, dalam hal ini berdasarkan field type. Selain itu, digunakan operator $sum untuk menghitung jumlah produk dalam setiap kelompok tipe. Contoh perintah db.product.aggregate([{$group : {_id : "$type", num_product: {$sum:1}}}]) akan mengelompokkan semua dokumen di koleksi product berdasarkan nilai type, dan menghitung total produk untuk setiap tipe tersebut. Hasilnya menunjukkan bahwa terdapat 2 produk bertipe ‘phone’, 4 produk bertipe ‘accessory’, 3 produk bertipe ‘service’, serta masing-masing 1 produk untuk tipe ‘tv’ dan ‘warranty’. Ini memberikan gambaran jumlah produk berdasarkan kategori tipe secara ringkas dan terstruktur.



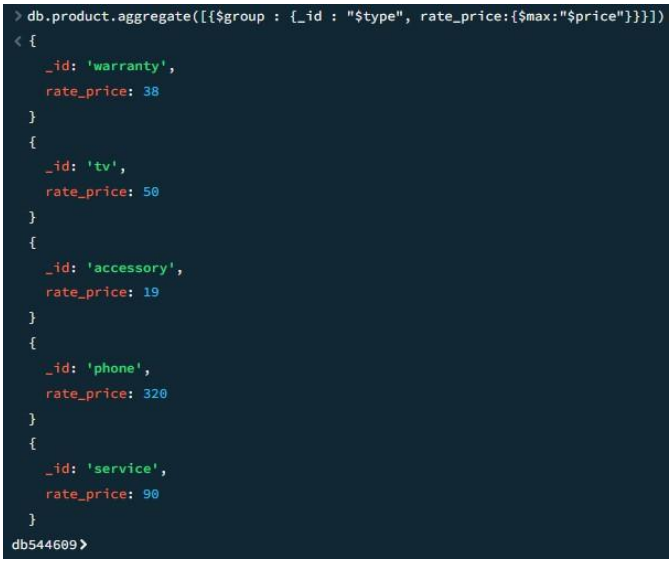
Pembahasan:

Dalam hasil query tersebut, digunakan fungsi aggregate() dengan tahap $group untuk menghitung rata-rata harga dari setiap tipe produk. Perintah db.product.aggregate([{$group : {_id : "$type", rate_price: {$avg: "$price"}}}]) mengelompokkan dokumen berdasarkan field type, kemudian menghitung nilai rata-rata dari field price untuk setiap kelompok menggunakan operator $avg. Hasilnya menunjukkan bahwa rata-rata harga produk bertipe ‘accessory’ adalah 13.875, untuk tipe ‘phone’ sekitar 260, tipe ‘service’ sebesar 63.33, ‘warranty’ sebesar 38, dan ‘tv’ sebesar 50. Pemanfaatan proses ini memudahkan analisis harga rata-rata berdasarkan kategori tipe produk dalam koleksi.



Pembahasan:

Dalam hasil query tersebut, digunakan fungsi aggregate() dengan tahap $group untuk menentukan harga minimum dari masing-masing tipe produk. Perintah db.product.aggregate([{$group : {_id : "$type", rate_price: {$min: "$price"}}}]) mengelompokkan dokumen berdasarkan field type, kemudian mencari nilai terkecil dari field price dalam setiap kelompok menggunakan operator $min. Hasilnya menunjukkan bahwa harga terendah untuk produk bertipe ‘phone’ adalah 200, ‘accessory’ adalah 12, ‘service’ adalah 40, ‘tv’ adalah 50, dan ‘warranty’ adalah 38. Penggunaan fungsi ini sangat membantu untuk mengetahui batas harga paling rendah dari setiap kategori produk dalam koleksi, sehingga dapat digunakan untuk analisis harga diskon, penetapan harga minimum, atau pengawasan harga pasar.



Pembahasan:

Gambar tersebut menampilkan hasil query MongoDB yang menggunakan fungsi aggregate() dengan operator $max. Sintaks db.product.aggregate([{$group : {_id : "$type", rate_price: {$max: "$price"}}}]) berfungsi mengelompokkan dokumen berdasarkan nilai dari field type, kemudian mencari harga tertinggi ($max) dari field price di dalam setiap grup tersebut. Hasilnya menunjukkan bahwa harga tertinggi untuk produk bertipe ‘warranty’ adalah 38, ‘tv’ adalah 50, ‘accessory’ adalah 19, ‘phone’ adalah 320, dan ‘service’ adalah 90. Penggunaan operasi agregasi ini sangat bermanfaat untuk mengidentifikasi batas atas harga dalam berbagai kategori produk, misalnya untuk analisis harga tertinggi, penetapan range harga, atau mengidentifikasi produk-produk mahal dalam setiap tipe.



Pembahasan:

Gambar tersebut menampilkan hasil query MongoDB yang menggunakan fungsi aggregate() dengan operator $push. Perintah db.product.aggregate([{$group : {_id : "$type", name_product: {$push: "$name"}}}]) berfungsi untuk mengelompokkan data berdasarkan field type dan mengumpulkan semua nilai dari field name dalam setiap kelompok ke dalam sebuah array menggunakan $push. Hasilnya memperlihatkan daftar nama produk dalam tiap kategori, misalnya kategori ‘phone’ berisi tiga produk seperti ‘AC3 Phone’ dan ‘AC7 Phone’, kategori ‘accessory’ memiliki empat item termasuk ‘AC3 Case Black’, dan seterusnya. Penggunaan $push ini sangat berguna untuk mengumpulkan semua data terkait dalam satu array untuk analisis atau tampilan yang lebih lengkap per kategori.



Pembahasan:

Gambar tersebut menampilkan hasil query MongoDB yang menggunakan operator $addToSet. Perintah db.product.aggregate([{$group : {_id : "$type", name_product: {$addToSet: "$name"}}}]) berfungsi untuk mengelompokkan data berdasarkan field type dan mengumpulkan nama produk ke dalam sebuah array tanpa duplikasi menggunakan $addToSet. Dibandingkan dengan $push, yang dapat memasukkan nilai yang sama berulang kali, $addToSet memastikan hanya nilai unik yang disimpan dalam array. Contohnya, dalam kategori ‘accessory’, hanya muncul empat nama produk yang berbeda, tidak ada yang berulang. Pendekatan ini sangat efisien untuk mendapatkan daftar produk yang bersih dan tidak redundan sesuai kategori.



Pembahasan:

Gambar tersebut menunjukkan hasil query MongoDB yang menggunakan operator $last. Perintah db.product.aggregate([{$group : {_id : "$type", name_product : {$last: "$name"}}}]) berfungsi untuk mengelompokkan data berdasarkan field type, kemudian mengambil nilai terakhir dari field name dalam setiap grup sesuai urutan dokumen saat ini di koleksi. Misalnya, untuk tipe ‘phone’, produk terakhir adalah ‘AC7 Phone’, dan untuk ‘accessory’ adalah ‘AC3 Case Red’. Karena $last mengambil entri terakhir dalam urutan dokumen, hasilnya sangat bergantung pada urutan input, dan bukan berdasarkan urutan tertentu seperti alfabetis atau harga. Jadi, urutan dokumen saat penyimpanan atau pengambilan bisa mempengaruhi hasilnya.



Pembahasan:

Gambar tersebut menunjukkan hasil query MongoDB yang menggunakan operator $first. Perintah db.product.aggregate([{$group : {_id : "$type", name_product: {$first : "$name"}}}]) berfungsi untuk mengelompokkan data berdasarkan field type, lalu mengambil nilai pertama dari field name dalam setiap grup sesuai urutan dokumen dalam koleksi. Contohnya, untuk tipe ‘phone’, produk pertama yang muncul adalah ‘AC3 Phone’, dan untuk ‘accessory’ adalah ‘AC3 Series Charger’. Karena $first meninjau urutan dokumen yang ada, hasilnya bergantung pada urutan input dan bukan berdasarkan urutan alfabetis atau harga. Operator ini sangat berguna untuk mendapatkan entri awal dari setiap kategori tanpa perlu menyortir data ulang.

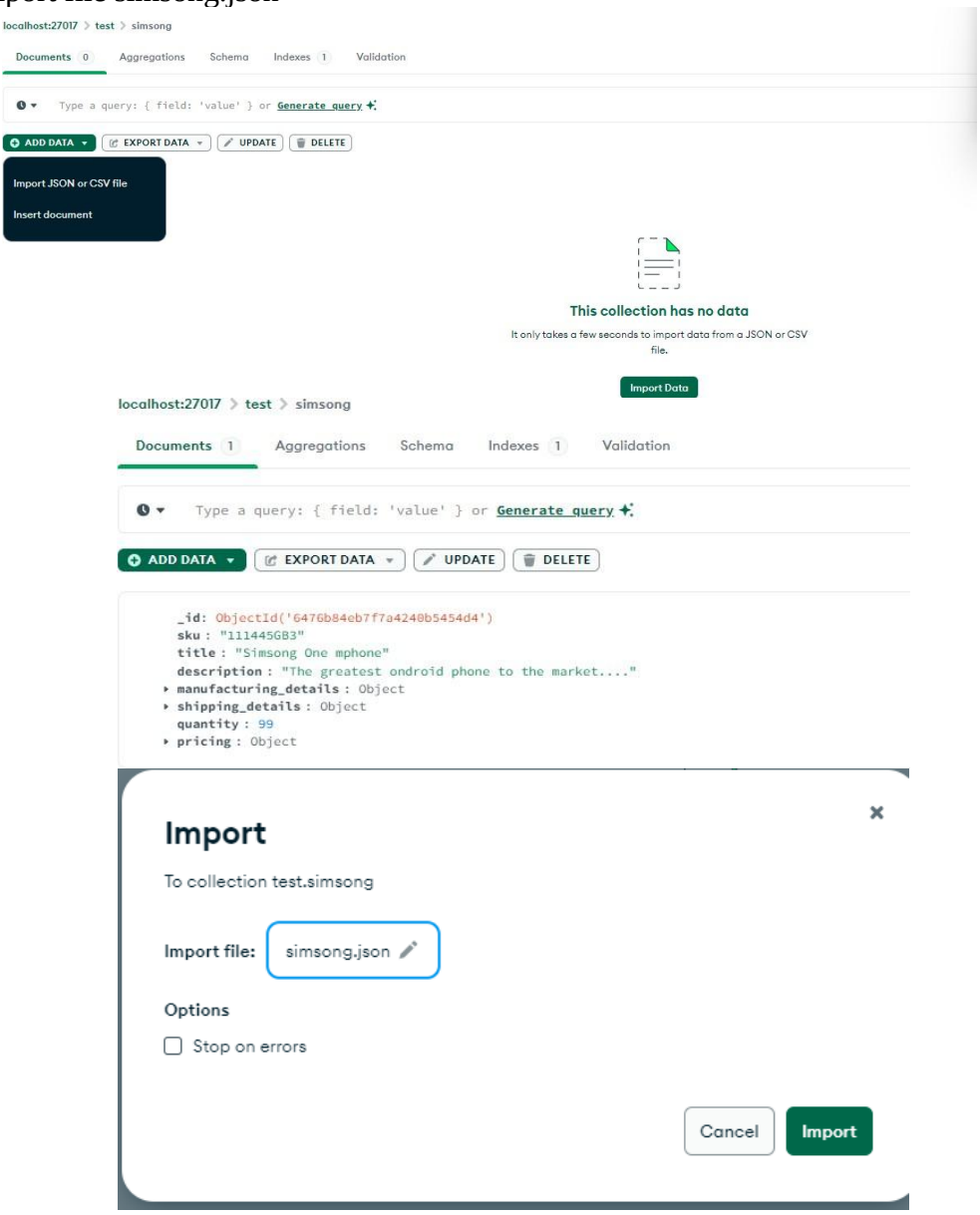
```
> db.product.aggregate([{$match :{"type":"service"}}, {$group : {_id : "$type", num_product:{$sum:1}}}])
< {
    _id: 'service',
    num_product: 3
  }
db544609>
```

Pembahasan:

Query ini berfungsi untuk menghitung berapa banyak dokumen dalam koleksi product yang memiliki nilai "service" pada field type. Menggunakan tahap $match, query menyaring dokumen sehingga hanya yang bertipe "service" yang diproses selanjutnya. Setelah itu, tahap $group mengelompokkan data berdasarkan type dan menghitung jumlah dokumen dalam setiap grup dengan memakai $sum. Hasilnya menunjukkan bahwa dalam koleksi terdapat 3 produk yang bertipe "service".

Tugas Simsong

a. Simsong.inc meluncurkan produk terbaru oneDroid ke pasar, untuk mendukung strategi pemasaran bangunlah suatu pemodelan e-commerce untuk stock produk.
   - Desain skema untuk e-commerce
   - Import file simsong.json



File simsong.json yang sudah didwonload, diimpor ke database. File ini berisi data produk. Setiap produk memiliki atribut seperti title, sku, description, quantity dan pricing untuk menggambarkan struktur kategori produk.

1. Semua produk mempunyai kategori pada collection belum terdapat field kategori,gunakan update query untuk menambah kategori produk

   Categories: [mobile/15G, mobile/fm] dengan sku: {“111445GB3”}

```
> db.simsong.update({"sku":"111445GB3"}, {$set: {"categories":['mobile/15G','mobile/fm']}})
< DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
test >
```

Pembahasan:

Query update ini digunakan untuk menambahkan sebuah field baru bernama categories ke dalam dokumen di koleksi simsong. Dokumen yang akan diubah dipilih berdasarkan nilai sku yang sama dengan "111445GB3". Field categories yang ditambahkan berisi array yang terdiri dari dua string: 'mobile/15G' dan 'mobile/fm', sehingga produk tersebut bisa diklasifikasi ke dalam lebih dari satu kategori secara bersamaan.

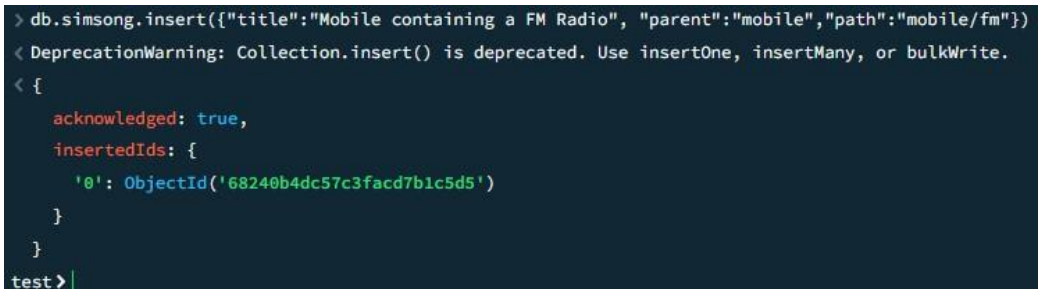
2. Untuk mendukung efisien pencarian query penggunaan field kategori, tambahkan indeks pada field kategori pada koleksi produk:

```
> db.simsong.ensureIndex({"categories":1})
< [ 'categories_1' ]
test >
```

Pembahasan:

Perintah db.simsong.ensureIndex({"categories": 1}) digunakan untuk membuat indeks menaik pada field categories di koleksi simsong. Pembuatan indeks ini sangat penting untuk mempercepat proses pencarian data, terutama saat melakukan query berdasarkan kategori produk. Dengan adanya indeks, MongoDB dapat melakukan pencarian dengan lebih efisien karena tidak perlu memindai seluruh koleksi, melainkan menggunakan struktur indeks yang lebih terorganisir. Hasil dari perintah tersebut menunjukkan bahwa indeks berhasil dibuat dan secara otomatis diberi nama categories_1, yang menunjukkan bahwa indeks tersebut diurutkan secara menaik.

3. Untuk dapat menyediakan daftar semua produk dalam suatu kategori, ubahlah model data dengan kumpulan dokumen untuk setiap kategori. Dalam koleksi ini, setiap dokumen mewakili kategori dan berisi path untuk kategori dalam struktur data dokumen



Pembahasan:

Perintah yang terlihat dalam gambar adalah contoh penggunaan metode insert() di MongoDB untuk menambahkan dokumen baru ke dalam koleksi simsong. Dokumen yang dimasukkan memiliki tiga field: title dengan nilai "Mobile containing a FM Radio", parent dengan nilai "mobile", dan path dengan nilai "mobile/fm". Setelah perintah dijalankan, MongoDB akan mengembalikan hasil yang menunjukkan acknowledged: true, menandakan bahwa operasi sukses, serta insertedIds yang berisi ID unik (ObjectId) dari dokumen yang baru dimasukkan.

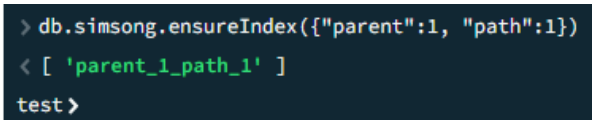
4. Buatlah index mendukung efisien pencarian query penggunaan field parent dan path dengan fungsi decrement



Pembahasan:

Perintah dalam gambar menunjukkan bagaimana menambahkan dokumen ke koleksi simsong di MongoDB menggunakan metode insert. Dokumen yang disisipkan terdiri dari tiga field: title, parent, dan path, masing-masing dengan nilai yang sesuai.

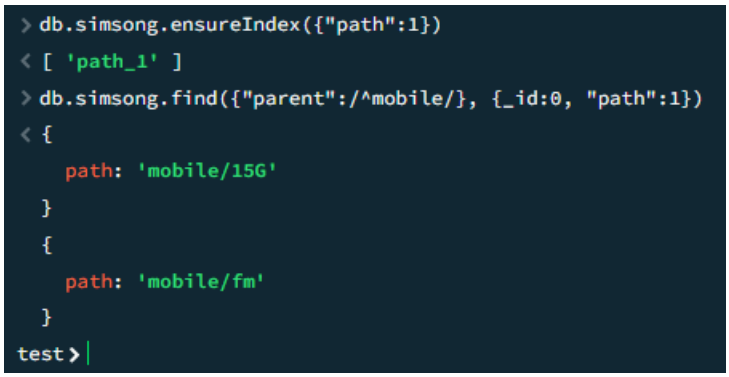
5. Buatlah index mendukung efisien pencarian query penggunaan field path dengan fungsi decrement



Pembahasan:

Gambar tersebut menampilkan bahwa indeks gabungan berhasil dibuat pada koleksi simsong untuk field parent dan path dengan menggunakan metode ensureIndex(). Indeks ini disusun dalam urutan menaik, sebagaimana ditunjukkan oleh nilai 1 pada masing-masing field. Nama default yang diberikan untuk indeks ini adalah parent_1_path_1. Tujuan dari pembuatan indeks ini adalah untuk meningkatkan efisiensi dalam menjalankan query yang sering melakukan filter atau pengurutan berdasarkan kedua field tersebut. Dengan adanya indeks ini, MongoDB bisa menavigasi dokumen dengan lebih cepat tanpa harus melakukan pencarian menyeluruh ke seluruh koleksi, sehingga secara signifikan mempercepat waktu respon terhadap permintaan data terkait struktur atau hierarki kategori dokumen.

6. Terapkan fungsi index pada pencarian data



Pembahasan:

Perintah db.simsong.find({"parent":/^mobile/}, {_id:0, "path":1}) berfungsi untuk mencari semua dokumen di koleksi simsong yang memiliki nilai parent yang diawali dengan kata "mobile" menggunakan regular expression. Hanya field path yang ditampilkan dalam hasilnya, tanpa menampilkan _id. Hasil dari query ini menunjukkan dua dokumen dengan path berisi 'mobile/15G' dan 'mobile/fm', yang mengindikasikan bahwa pencarian tersebut berhasil dan didukung oleh indeks yang sebelumnya telah dibuat, sehingga proses pencariannya lebih efisien.

F. Kesimpulan

1. Dari praktikum ini, dapat disimpulkan bahwa fitur agregasi di MongoDB sangat penting untuk melakukan analisis data secara kompleks dan efisien. Melalui tahapan-tahapan seperti $group, $match, $sort, dan operator agregasi seperti $sum, $avg, $min, $max, $push, $addToSet, $first, serta $last, pengguna dapat mengelompokkan, menyaring, mengurutkan, dan menghitung statistik data sesuai kebutuhan. Praktikum ini juga menyoroti pemanfaatan fungsi seperti skip() dan limit() yang membantu dalam mengelola hasil query, terutama untuk fitur pagination atau pembatasan jumlah data yang ditampilkan.
2. Selain itu, praktikum ini memperdalam pemahaman tentang pentingnya indeks dalam meningkatkan kecepatan pencarian data. Pembuatan indeks pada field tertentu seperti categories, parent, dan path terbukti mempercepat proses pencarian, karena MongoDB tidak perlu melakukan pemindaian penuh koleksi (full collection scan), melainkan langsung mengakses data melalui indeks (IXSCAN). Jenis-jenis indeks seperti indeks satu field, indeks gabungan, dan indeks multikey juga dipelajari untuk memenuhi berbagai kebutuhan query.
3. Secara keseluruhan, praktikum ini memberi wawasan tentang konsep agregasi dan indeks dalam MongoDB, serta penerapannya dalam skenario nyata, seperti pengelolaan produk e-commerce. Dengan memahami teknik-teknik ini, mahasiswa dapat mengoptimalkan performa basis data dan melakukan pengolahan data secara lebih efektif.

Daftar Pustaka

Fungsi Agregat SQL SUM, SQL COUNT, dan SQL AVG 2025. (n.d.). Retrieved from https://www.revou.co/panduan-teknis/sql-sum

Mulya, M. P. (2023). Agregasi Data pada SQL. Retrieved from https://marioprasetyamulya85.medium.com/agregasi-data-pada-sql-31df71188355

Hans, R. (2023). Belajar Fungsi Group By & Aggregate di SQL. Retrieved from https://dqlab.id/belajar-fungsi-group-by-and-aggregate-di-sql